Report:

“BEHAVIORAL SEGMENTATION FOR TARGETED MARKETING”

Problem Id: 10

Student Name: Akshita Bahuguna(2028387)

Student Name: Avika Dixit(2028404)

Student Name:Vanya Rathi(2028514)

Section : L

- **Problem Description:**

In today's competitive retail market, understanding customer behavior is essential for businesses to improve marketing strategies and increase profitability. Customers differ in their purchasing power, spending habits, and shopping frequency. Traditional marketing approaches often fail because they treat all customers the same, leading to inefficient targeting and reduced business performance.

This project focuses on behavioral segmentation, where customers are grouped based on their purchasing behavior such as Annual Spending and Purchase Frequency. By identifying patterns in customer data, businesses can create personalized marketing strategies for different customer groups.

In Phase 1, the theoretical design of the machine learning pipeline was completed. Concepts such as data preprocessing, feature scaling, clustering, and visualization were studied.

In Phase 2, a synthetic dataset of 1000 customers was generated and preprocessed. Missing values were handled, outliers were removed using the IQR method, categorical variables were encoded, and numerical features were scaled.

In Phase 3, the K-Means Clustering algorithm is applied to segment customers into distinct groups. The StandardScaler is used to normalize the data, and the Elbow Method is used to determine the optimal number of clusters.

This project helps businesses identify customer personas and improve targeted marketing strategies.

Steps of Implementation:

- ## Data Collection / Generation:

A synthetic dataset of 1000 customers was created containing features such as:

1. Annual Spending
2. Purchase Frequency
3. Income
4. Gender
5. Preferred Category
6. Last Purchase Amount

The dataset was stored in CSV format and loaded using Pandas.

- ## Exploratory Data Analysis (EDA):

1. Used head(), info(), describe() to understand data
2. Identified missing values and inconsistencies
3. Visualized data using scatter plots and histograms

- ## Data Preprocessing:

➔ **Missing Values Handling:**
    1. Mean for numerical features
    2. Mode for categorical features

➔ **Outlier Detection:**
    1. Used IQR method
    2. Removed extreme values

➔ **Encoding:**
    1. Label Encoding for categorical variables

➔ **Feature Scaling:**

1. StandardScaler used for normalization

- ## Feature Selection:

In this step, only the most relevant features were selected from the dataset to improve clustering quality and reduce unnecessary complexity.

• Removing Redundant Features:

➔ Features that do not contribute to customer behavior were removed.
➔ Columns like id, name, etc., were excluded as they do not affect clustering.
➔ This helps in improving clustering accuracy and reducing noise.

• Selecting Important Features:

- Based on project objective, the following features were selected:

1. Annual Spending (spending_score)
2. Purchase Frequency

- These features directly represent customer purchasing behavior.

Code:

```
X = df[['spending_score', 'purchase_frequency']]
```

- ## Model Development:

In this step, the **K-Means Clustering algorithm** was implemented to group customers into different segments.

- **Applying StandardScaler:**
  ➔ Since K-Means is distance-based, scaling is necessary.
  ➔ StandardScaler converts data to mean = 0 and standard deviation = 1.

Code:

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

• **Applying K-Means Clustering:**

➔ K-Means groups similar data points into clusters.
➔ Each cluster is represented by a centroid.

Code:

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=4, random_state=42)

clusters = kmeans.fit_predict(X_scaled)

df['Cluster'] = clusters
```

- ## Model Evaluation:

Since clustering is an unsupervised learning method, traditional metrics like accuracy are not used.

1. **Elbow Method (WCSS):**

➔ WCSS (Within Cluster Sum of Squares) measures cluster compactness
➔ Lower WCSS = better clustering

Code:

```
wcss = []

for i in range(1, 11):

    kmeans = KMeans(n_clusters=i, random_state=42)

    kmeans.fit(X_scaled)

    wcss.append(kmeans.inertia_)
```

2. **Plotting Elbow Graph:**

Code:

```
import matplotlib.pyplot as plt

plt.plot(range(1, 11), wcss)

plt.xlabel("Number of Clusters")

plt.ylabel("WCSS")

plt.title("Elbow Method")

plt.show()
```

- ## Visualization

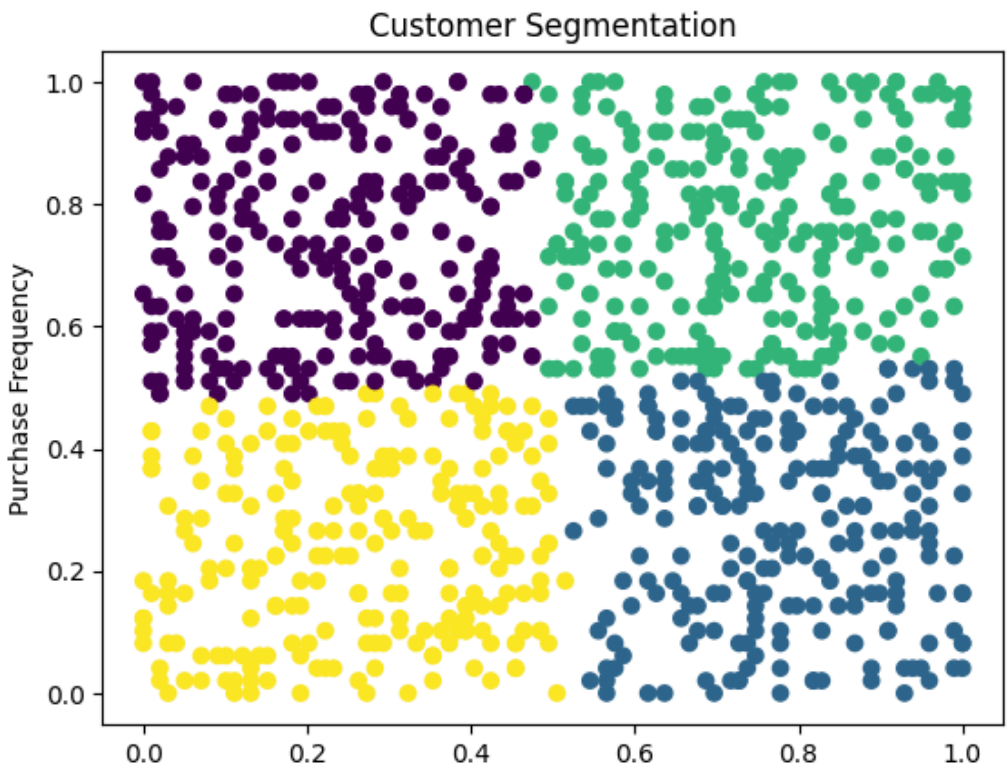
Visualization helps in understanding how customers are grouped.

1-Scatter Plot:

Code:

```
kmeans = KMeans(n_clusters=4, random_state=42)
df['Cluster'] = kmeans.fit_predict(X_scaled)

plt.scatter(df['spending_score'], df['purchase_frequency'], c=df['Cluster'])
plt.xlabel("Annual Spending")
plt.ylabel("Purchase Frequency")
plt.title("Customer Segmentation")
plt.show()
```



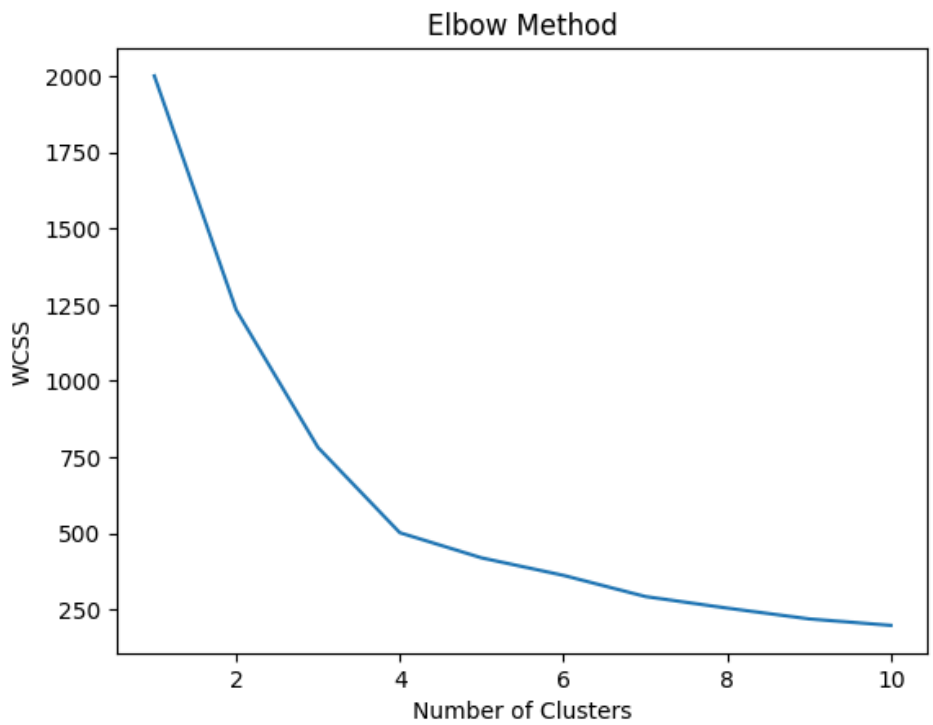
2-Elbow Method:

Code:

```
wcss = []

for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.plot(range(1, 11), wcss)
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```



- **Final Segmentation:**

Customers were divided into 4 segments:

➔ **Cluster 0:** High Spending, High Frequency (Premium Customers)
➔ **Cluster 1:** Low Spending, Low Frequency (Low-value Customers)
➔ **Cluster 2:** High Spending, Low Frequency (Occasional Buyers)
➔ **Cluster 3:** Low Spending, High Frequency (Regular Budget Customers)

• Output Analysis:

Graphs used:

- Elbow Method Graph (WCSS vs K)
- Scatter Plot (Customer Segmentation)

These visualizations clearly show grouping of customers into different clusters.

Expected Modules/ Libraries to be used :

NumPy:

NumPy is a very important library that is used in project for handling numbers quickly and easily. It does the work with big lists of numbers (arrays and matrices), which is really helpful when dealing with a lot of customer data. NumPy makes it simple to do math like finding the average, variance, and standard deviation much faster than using normal Python loops. In this project, NumPy is used to generate random datasets, modify the data during preprocessing, and assist with scaling and normalisation. Since NumPy can do calculations on whole arrays at once, it saves a lot of time and makes working with data much more efficient.

Pandas:

Pandas is a data analysis and manipulation library that plays a central role in organising and processing datasets. It introduces the DataFrame structure, which allows data to be stored in a tabular format with labelled rows and columns, making it easy to work with structured customer information such as age, gender, spending, and purchase frequency. In this project, Pandas is used extensively for data cleaning tasks such as handling missing values (using mean, median, or mode imputation), removing duplicate entries, and transforming raw data into a usable format. It also supports advanced operations like grouping, aggregation, and filtering, which help in analyzing customer behaviour patterns. Additionally, it simplifies the process of encoding categorical variables and exporting/importing datasets in formats like CSV.

Matplotlib:

Matplotlib is a comprehensive data visualization library used to represent data graphically, which is important for understanding patterns and validating results in the project. It enables the creation of various types of plots such as scatter plots, line graphs, bar charts, and specifically the elbow curve used in clustering analysis. In this project, scatter plots help visualize how customers are grouped based on features like annual spending and purchase frequency, providing intuitive insights into different customer segments. The elbow method graph, created using Matplotlib, helps determine the optimal number of clusters by plotting the Within-Cluster Sum of Squares (WCSS)

against different values of K. These visual tools not only enhance interpretability but also support decision-making by clearly presenting patterns and relationships in the data.

Scikit-learn:

Scikit-learn is a widely used machine learning library that provides efficient and easy-to-implement tools for data preprocessing, model building, and evaluation. In this project, it is primarily used for implementing StandardScaler, which standardizes the dataset by transforming features to have zero mean and unit variance—an essential step for distance-based algorithms like K-Means. The K-Means clustering algorithm from Scikit-learn is used to segment customers into different groups based on behavioral similarities such as spending habits and purchase frequency. The library also provides utilities for splitting data into training and testing sets, as well as evaluation techniques to validate model performance.

Expected Evaluation metrics and Plots to be used:

- ## WCSS (Within Cluster Sum of Squares)

WCSS measures the total distance between each data point and its corresponding cluster centroid. It indicates how compact the clusters are. Lower WCSS values represent better clustering performance.

- ## Elbow Method

The Elbow Method is used to determine the optimal number of clusters (K). It works by plotting WCSS values against different values of K.

- The point where the graph bends (forms an “elbow”) is considered the optimal number of clusters.
- This helps in selecting the most suitable number of customer segments.

- ## Scatter Plot Visualization

Scatter plots are used to visually represent customer clusters.

- Each point represents a customer
- Different colors represent different clusters
- Helps in understanding separation between customer groups

- ## Elbow Graph (WCSS vs Number of Clusters)

This graph visually represents how WCSS changes with different values of K.

- Used to justify the selection of optimal clusters
- Helps in improving model decision-making

Required topics from the subject:

Data Structures (Arrays & DataFrames):

Used to store and manage customer data like Annual Spending and Purchase Frequency using NumPy arrays and Pandas DataFrames**.**

Functions:

Used to structure the program into reusable blocks such as data generation, scaling, and clustering steps.

File Handling:

Used to save and load the dataset (e.g., CSV files) for future use and analysis**.**

Loops & Conditionals:

Used in implementing the Elbow Method (loop over different cluster values) and making decisions in the program.

Libraries & Modules:

Used in implementing the Elbow Method (loop over different cluster values) and making decisions in the program.

Platform Required:

Jupyter Notebook:

Used as the primary development environment for writing, executing, and interactively visualising the Python Code.

Python (Anaconda Distribution):

Provides all required libraries and package management needed for data analysis and machine learning.

Books and Link Sources:

1. "Hands-On Machine Learning with Scikit-Learn" by Aurélien Géron
2. Python for Data Analysis" by Wes McKinney
3. Scikit-learn Documentation (https://scikit-learn.org/)
4. Kaggle (https://www.kaggle.com/)